

713 小结

模板是 C++ 语言与众不同的特性，也是标准库的基础。一个模板就是一个编译器用来生成特定类类型或函数的蓝图。生成特定类或函数的过程称为实例化。我们只编写一次模板，就可以将其用于多种类型和值，编译器会为每种类型和值进行模板实例化。

我们既可以定义函数模板，也可以定义类模板。标准库算法都是函数模板，标准库容器都是类模板。

显式模板实参允许我们固定一个或多个模板参数的类型或值。对于指定了显式模板实参的模板参数，可以应用正常的类型转换。

一个模板特例化就是一个用户提供的模板实例，它将一个或多个模板参数绑定到特定类型或值上。当我们不能（或不希望）将模板定义用于某些特定类型时，特例化非常有用。

最新 C++ 标准的一个主要部分是可变参数模板。一个可变参数模板可以接受数目和类型可变的参数。可变参数模板允许我们编写像容器的 `emplace` 成员和标准库 `make_shared` 函数这样的函数，实现将实参传递给对象的构造函数。

术语表

类模板 (class template) 模板定义，可从它实例化出特定的类。类模板的定义以关键字 `template` 开始，后跟尖括号对 `<` 和 `>`，其内为一个用逗号分隔的一个或多个模板参数的列表，随后是类的定义。

默认模板实参 (default template argument) 一个类型或一个值，当用户未提供对应模板实参时，模板会使用它。

显式实例化 (explicit instantiation) 一个声明，为所有模板参数提供了显式实参。714 实例 (instantiation) 编译器从模板生成用来指导实例化过程。如果声明是 `extern` 的，模板将不会被实例化；否则，模板将利用指定的实参进行实例化。对每个 `extern` 模板声明，在程序中某处必须有一个非 `extern` 的显式实例化。

显式模板实参 (explicit template argument) 在一个函数调用中或定义模板类类型时，由用户提供的模板实参。显式模板实参在紧跟在模板名的尖括号对中给出。

函数参数包 (function parameter pack) 表示零个或多个函数参数的参数包。

函数模板 (function template) 模板定义，可从它实例化出特定函数。函数模板的定

义以关键字 `template` 开始，后跟尖括号对 `<` 和 `>`，其内为一个用逗号分隔的一个或多个模板参数的列表，随后是函数的定义。

实例化 (instantiate) 编译器处理过程，用实际的模板实参来生成模板的一个特殊实例，其中参数被替换为对应的实参。当函数模板被调用时，会自动根据传递给它的实参来实例化。而使用类模板时，则需要我们提供显式模板实参。

实例 (instantiation) 编译器从模板生成的类或函数。

成员模板 (member template) 本身是模板的成员函数。成员模板不能是虚函数。

非类型参数 (nontype parameter) 表示值的模板参数。非类型模板参数的实参必须是常量表达式。

包扩展 (pack expansion) 处理过程，将一个参数包替换为其中元素的列表。

参数包 (parameter pack) 表示零个或多个参数的模板或函数参数。

部分特例化 (partial specialization) 类模板的一个版本，其中指定了某些但不是所